Que es este diseño y que cambió

Este documento presenta el diseño del sistema del Club Deportivo Séneca (CDS) para la Entrega 2 del Proyecto 1. El diseño tiene 64 elementos: 59 clases (11 abstractas y 48 concretas) y 5 enumeraciones. Con el documento van tres archivos que cuentan exactamente lo mismo: el diagrama de clases detallado (UXF), el diagrama de alto nivel (UXF) y las imágenes de los dos. El de alto nivel es el detallado sin atributos ni métodos y con las clases más juntas, no tiene ninguna clase ni relación que el detallado no tenga.

El diseño en pocas palabras:

- **ClubDeportivo** es la raíz de todo lo que se guarda: tiene las listas de usuarios, deportes, equipos, canchas, reservas, entrenamientos, partidos, competencias, artículos, sugerencias, turnos y solicitudes. No decide reglas de los temas.
- **Cinco gestores sin estado**, uno por tema: GestorUsuarios, GestorDeportivo, GestorReservas, GestorVentas y GestorTurnos. Reciben el club en el constructor y no se conocen entre sí.
- **Las clases del dominio** guardan sus datos y deciden las reglas que solo dependen de ellas. Por ejemplo, Equipo decide si la camiseta está libre y Cancha decide si se cruza con otra ocupación.
- **PersistenciaClub** guarda y carga todo el club en un solo archivo.

En la Entrega 1 el UML era un modelo de dominio, sin gestores, sin persistencia y con algunas decisiones que al desarrollar los requerimientos no alcanzaron. Lo que cambió:

- **Tienda y Cafetería desaparecen como clases** con arreglos de artículos. Ahora hay un solo Articulo con un mapa de cantidades por Ubicacion (BODEGA, TIENDA, CAFETERIA). En la Entrega 1 ya habíamos notado que faltaba la ubicación en el inventario, y pues ahora quedó. Por eso también desaparece ArticuloTienda: el IVA pasa a TransTienda, y el impuesto al consumo y la propina a TransCafeteria.
- **Los puntos de fidelidad, el estado activo y las transacciones pasan de Jugador a Usuario**, porque los RF-SOC-12 a RF-SOC-16 también aplican para los empleados.
- **La suspensión ya no es un dato calculado del Jugador.** Vive en InfoJugador (partidos internos) y en JugadorNomina (competencias), que es una clase nueva.
- **Los partidos entre equipos suben a AbstractPartidoEquipos** (marcador y alineación), así PartidoIndividual no carga con un marcador de equipos.
- **Entrenamiento** guarda fecha, horaInicio, horaFin, su cancha y si está activo, en vez de hora y duración. **Reserva** pierde la modalidad (la cantidad de jugadores ya la define) y gana el estado activa.
- **Cancha guarda sus Ocupaciones** (clase nueva) y el Club es el que asigna la cancha. Así las reservas y los entrenamientos usan el mismo mecanismo y no se pueden cruzar.
- **SugerenciaArticulo** ya no apunta al Admin que la resuelve: el Club guarda la lista y GestorVentas la resuelve. Además guarda sus alérgenos (RN-13).
- **Nacen** los 5 gestores, PersistenciaClub y las 5 enumeraciones (Categoria, Nivel, EstadoAsistencia, Ubicacion y EstadoSolicitud).

Qué cambió frente al borrador de diseño del grupo

El borrador (disenioP1_g11) fue la base. Lo que se mantiene es la arquitectura: Club como raíz, gestores por tema sin estado y serialización en un solo archivo. Lo que se cambió:

- **Se quita la capa Persona y Rol** (Persona, Rol, RolJugador, RolEmpleado, RolEntrenador y RolFisio). Queda la jerarquía Usuario (abstracta) con Jugador, AbstractEmpleado (Entrenador y Fisio) y Admin. Login, clave, nombre, estado activo y puntos viven en Usuario. La consecuencia es que un empleado no es a la vez jugador; la excepción de RN-10 se resuelve con el atributo tambienEntrenador de Jugador.
- **Se corrigen los nombres**: Equipos pasó a Equipo, infoJugador a InfoJugador, Etrenamiento a Entrenamiento, PartidoIdividual a PartidoIndividual, Depconjunto a DepConjunto, y bebida y snack a Bebida y Snack.
- **Cancha** deja de tener agregarReserva, cancelarReserva y agregarEntrenamiento. Ahora tiene ocupar, liberar, getOcupaciones, tieneCapacidadPara y permiteBebidaCaliente, y el Club suma asignarCancha, liberarCancha, contarOcupadas y bloqueValido.
- **reservar y crearEntrenamiento piden el deporte y no una cancha**: el Club decide cuál cancha se asigna. El deporte además guarda cuántas instalaciones simultáneas permite (maxSimultaneas, RN-09).
- **Se agregan los métodos que faltaban** para que cada RF tenga dueño: reabastecer en Articulo, reabastecerTienda en GestorVentas, cancelar y estaActiva en Reserva, registrarAsistencia en GestorReservas, getEquiposDe en GestorDeportivo, entre otros.

Ajustes finales de esta versión

Estos son los cambios que hicimos al diagrama detallado que ya habíamos visto, para que quede consistente con los RF y sin relaciones circulares:

1. **Se quitan las relaciones en los dos sentidos.** Ahora solo apunta el que contiene: Equipo a InfoJugador, AbstractCompetencia a PartidoCompetencia y a ParticipacionCompe, Fase a ParticipacionFase, Jugador a InscripcionDisciplina, Entrenamiento a AsistenciaEntreno y Usuario a AbstractTransaccion. También se quita la flecha de AbstractEstadisticas hacia AbstractPartido (el partido guarda las estadísticas por jugador, no al revés) y la de Admin hacia SugerenciaArticulo (ya la guarda el Club).
2. **GestorDeportivo** gana registrarAlineacion (RF-ENT-05), getEstadisticasDe (RF-SOC-09 y RF-ENT-10) y el método privado competenciaDe, que busca a qué competencia pertenece un partido.
3. **GestorVentas** gana compartirCodigo (RF-EMP-01) y sugerirArticulo recibe los alérgenos (RF-ENT-09).
4. **SugerenciaArticulo** guarda alergenos y **Jugador** guarda tambienEntrenador (RN-10).
5. **Las notas amarillas** ya no citan decisiones del borrador: dicen lo que hay (un solo archivo club.dat en la carpeta datos, fuera de src, y gestores sin datos).

Estereotipos y asignación de responsabilidades

Primero cada clase según lo que hace, y después, una por una, qué sabe y qué decide.

Information holders (mayormente datos propios): Usuario con sus tipos (Jugador, AbstractEmpleado, Entrenador, Fisio y Admin), InfoJugador, Reserva, Ocupacion, Entrenamiento, AsistenciaEntreno, InscripcionDisciplina, los partidos (AbstractPartido, AbstractPartidoEquipos, PartidoInterno, PartidoCompetencia y PartidoIndividual), las estadísticas de cada deporte, Fase, ParticipacionCompe, ParticipacionFase, JugadorNomina, TurnoEmpleado, SolicitudCambioTurno, SugerenciaArticulo, Articulo con Bebida y Snack, las transacciones (AbstractTransaccion, TransTienda y TransCafeteria) y Compras.

Structurers (colecciones de otros objetos, con alguna decisión sencilla): ClubDeportivo (todas las colecciones del sistema), Equipo (sus jugadores con posición y camiseta), Cancha (sus ocupaciones), AbstractCompetencia (sus participaciones y partidos, y Ascun también sus fases) y Usuario (sus transacciones). Equipo, Cancha y AbstractCompetencia además son controladores de las reglas que solo usan sus datos.

Coordinadores y controladores: los cinco gestores. Cada uno coordina el flujo de su tema y decide cuando una regla cruza varios objetos.

Service providers: los deportes (AbstractDeporte, DepConjunto, DepIndividual y sus seis hijos), que dan las reglas propias de cada deporte (mínimo de jugadores, puntos, capacidad y estadísticas), y las enumeraciones Categoria y Nivel (admiteEdad y esAdyacente). No creamos calculadores aparte para el ranking, el precio o la tabla de posiciones: la lógica es corta y vive en PartidoIndividual, AbstractTransaccion y AbstractCompetencia.

Interfacers: PersistenciaClub, que lee y escribe el archivo.

Qué sabe y qué decide cada clase

Estructura y coordinación

ClubDeportivo: Es la raíz de todo lo que guarda el sistema y el único que sabe qué canchas están ocupadas. Sabe el horario de atención y tiene las listas de usuarios, deportes, equipos, canchas, reservas, entrenamientos, partidos, competencias, artículos, sugerencias, turnos y solicitudes. Decide si un bloque horario es válido (bloqueValido), cuántas instalaciones de un deporte están ocupadas en un bloque (contarOcupadas) y cuál cancha se asigna (asignarCancha) o se libera (liberarCancha). No decide las reglas de ningún tema, eso es de los gestores.

GestorUsuarios: Es el que maneja todo el tema de los usuarios: autentica, registra jugadores y los da de baja. Sabe cuáles son los usuarios (a través del Club) y decide si se puede registrar a alguien (que el login no esté en uso y que no falten datos). Para autenticar busca al usuario por su login, le pide que verifique la clave y revisa que esté activo. Al dar de baja a un jugador lo marca inactivo, lo retira de sus equipos (libera el cupo) y de sus academias; desde ahí los demás gestores lo rechazan para convocatorias, alineaciones y reservas porque revisan estaActivo.

GestorDeportivo: Es el que maneja todo el tema deportivo: equipos, academias, rankings, competencias y partidos entre equipos. Sabe qué equipos, partidos y competencias hay (a través del Club) y decide si se pueden crear, inscribir o registrar: que un jugador no esté ya en otro equipo del mismo deporte, que

un equipo cumpla para una competencia, que un jugador se pueda alinear, que un resultado no se haya registrado antes. Lo que depende solo del equipo (cupo, camiseta, edad) lo decide el Equipo. Al registrar un resultado aplica las suspensiones por tarjetas y suma los puntos de la competencia.

GestorReservas: Es el que maneja todo el tema de las instalaciones. Hace las reservas, programa los partidos de práctica y crea los entrenamientos, porque los tres ocupan una cancha. Sabe qué reservas y qué entrenamientos hay (a través del Club) y decide si se pueden hacer cruzando datos de varios objetos: inscripción a la academia, nivel de los jugadores, bloque horario, máximo de instalaciones simultáneas, capacidad, bebida caliente, jugadores activos y convocados que sí pertenezcan al equipo o la academia. Al cancelar libera la cancha por medio del Club.

GestorVentas: Es el que maneja todo el tema de la tienda y la cafetería: las compras, los descuentos y los puntos de fidelidad, el inventario (mover y reabastecer), las sugerencias de artículos y los reportes de ventas. Va de la mano con el Club para el inventario. Decide si una venta se puede hacer (que haya cantidad y que no sea una bebida caliente para alguien dentro de una cancha de pádel o una mesa techada), qué descuento aplica (una sola promoción) y registra la transacción con descuento, impuesto y puntos.

GestorTurnos: Es el que maneja todos los turnos de los empleados. Sabe cuáles son los turnos y las solicitudes de cambio y decide si se pueden hacer: que el turno esté dentro del horario de atención y que en cada hora queden mínimo 2 entrenadores y 1 fisioterapeuta (cumpleCobertura, privado porque solo lo usa este gestor). Aprueba o rechaza las solicitudes y rechaza las que ya no están pendientes.

PersistenciaClub: Es la que se encarga de guardar y cargar el club. Recibe la ruta por parámetro y no decide nada del negocio: guarda el club completo con guardar y lo reconstruye con cargar. Sus métodos son estáticos porque no guarda estado.

Usuarios

Usuario (abstracta): Es la entidad que tiene los datos de cualquier usuario del sistema. Sabe su login, su clave, su nombre, si está activo, sus puntos de fidelidad y sus transacciones. Decide si la clave que le muestran es la suya (autenticar), si tiene puntos suficientes para redimir (redimirPuntos) y cuántos puntos gana por una compra (acumularPuntos, el 2% del valor). Los puntos y las transacciones están aquí y no en Jugador porque los empleados también compran.

Jugador: Tiene los datos propios del socio. Sabe su edad, sus inscripciones a academias y si además es entrenador (tambienEntrenador, para RN-10). Se puede retirar de una inscripción.

AbstractEmpleado, Entrenador y Fisio: AbstractEmpleado sabe su código de descuento y decide si es entrenador (esEntrenador, que cada hijo responde). El fisioterapeuta no tiene funcionalidades propias (SP-06), solo se diferencia por su rol, que se usa en la cobertura de turnos.

Admin: Solo guarda el rol. Como no es Jugador, RN-02 se cumple por tipos: reservar, inscribir y programar prácticas piden Jugador.

Equipos, deportes e inscripciones

Equipo: Es el que tiene la información de un equipo y de los jugadores inscritos. Sabe su nombre, su deporte (DepConjunto), su categoría, su cupo, si está activo y sus jugadores con su InfoJugador. Decide

si todavía cabe otro jugador, si la camiseta está libre, si la edad corresponde a su categoría (Categoria.admiteEdad) y si la posición sirve para su deporte. Al darse de baja se queda inactivo y desinscribe a todos.

InfoJugador: Guarda lo de un jugador dentro de un equipo: su posición, su camiseta y si está suspendido.

AbstractDeporte, DepConjunto, DepIndividual y sus hijos: Son los que tienen las reglas propias de cada deporte. AbstractDeporte sabe su nombre y cuántas instalaciones simultáneas se permiten (RN-09) y sabe crear las estadísticas que le tocan (crearEstadisticas). DepConjunto sabe el mínimo de jugadores (RN-19), los puntos de victoria, empate y derrota (RN-25), las posiciones válidas y si permite empate. DepIndividual decide si admite cierta cantidad de jugadores (RN-06 a RN-08). Futbol, Baloncesto, Voleibol, Tenis, Padel y TenisMesa solo dan los valores.

InscripcionDisciplina: Es la que tiene la información de un jugador en una academia. Sabe su deporte, su nivel, su ranking y el historial del ranking. Se crea en PRINCIPIANTE (SP-04) y al actualizar el ranking decide si el jugador sube de nivel.

Instalaciones

Cancha: Es la que tiene los datos de una instalación. Sabe su id, su deporte, si es techada, su capacidad y sus ocupaciones. Decide si está disponible en un bloque (que no se cruce con otra ocupación), si tiene capacidad para cierta cantidad de jugadores y si permite una bebida caliente (RN-12). Se ocupa y se libera.

Ocupacion: Guarda un bloque ocupado: fecha, hora de inicio y hora de fin. Decide si se cruza con otro bloque (seCruza).

Reserva: Tiene la información de una reserva. Sabe su fecha, su horario, la cantidad de jugadores, los jugadores, su cancha y si está activa. Se puede cancelar.

Entrenamiento: Es el que tiene los datos de una sesión. Sabe la fecha, el horario, el entrenador, la cancha, si está activo y los convocados. Decide que un jugador no quede convocado dos veces y se puede cancelar.

AsistenciaEntreno: Guarda lo que pasó con cada convocado. Sabe el jugador, el estado de asistencia, la nota y el comentario. Decide que la nota esté entre 1 y 5 (RN-22) y que solo se evalúe a quien asistió.

Partidos y competencias

AbstractPartido: Tiene la información de un partido. Sabe su fecha, su deporte y las estadísticas de cada jugador. Decide que las estadísticas sean del tipo del deporte (registrarEstadisticas).

AbstractPartidoEquipos, PartidoInterno y PartidoCompetencia: AbstractPartidoEquipos sabe el marcador y la alineación, y decide que el resultado no se registre dos veces. PartidoInterno sabe sus dos equipos. PartidoCompetencia sabe el equipo del club, el equipo externo, el lugar y su fase.

PartidoIndividual: Sabe su reserva (que le da la cancha, el horario y los jugadores) y decide cómo cambia el ranking de cada jugador después del partido (actualizarRankings, ELO simplificado).

Estadísticas (AbstractEstadisticas y sus hijas de fútbol, baloncesto, voleibol, tenis y pádel, y tenis de mesa): Solo guardan sus datos, que cambian según el deporte. Las tarjetas están en EstadisticasFutbol.

AbstractCompetencia, Ascun y TorneoCerros: Es la que tiene la información de una competencia. Sabe su nombre, el organizador, la temporada, la categoría, el deporte, la nómina mínima, sus participaciones y sus partidos. Decide si un equipo cumple para inscribirse y si ya está inscrito (RN-24) y quién es el goleador. Ascun además sabe sus fases; TorneoCerros es una liga de todos contra todos.

Fase, ParticipacionCompe, ParticipacionFase y JugadorNomina: Solo guardan sus datos. ParticipacionCompe sabe los puntos de un equipo en la competencia y su nómina. JugadorNomina guarda las amarillas acumuladas y si el jugador está suspendido dentro de la competencia (RN-20 y RN-23).

Tienda y cafetería

Artículo, Bebida y Snack: Articulo sabe su nombre, categoría, precio y cuántas unidades hay en cada Ubicacion. Decide si hay cantidad suficiente para mover o descontar. Snack sabe sus alérgenos (RN-13) y Bebida sabe si es caliente.

AbstractTransaccion, TransTienda y TransCafeteria: AbstractTransaccion sabe la fecha, el descuento, los puntos redimidos y sus compras, y decide el total (calcularTotal, que cada hija calcula a su manera). TransTienda suma el IVA del 19%. TransCafeteria suma el impuesto al consumo del 8% y la propina. El tipo de la transacción es el rubro para los reportes (RN-16).

Compras: Es una línea de la transacción: el artículo, la cantidad y el precio por unidad del momento de la venta.

SugerenciaArticulo: Tiene la información de un artículo que un empleado propone. Sabe el nombre, la categoría, el precio, los alérgenos (si trae, es un snack), el estado y el empleado que la propone.

Turnos

TurnoEmpleado y SolicitudCambioTurno: Solo guardan sus datos. El turno sabe su empleado, el día de la semana y su horario. La solicitud sabe la fecha, el estado, el empleado, el turno actual y el turno nuevo.

Soporte a los requerimientos funcionales y a las reglas del negocio

Todos los RF de los cuatro tipos de usuario tienen una clase responsable. La lista dice quién y con qué método. Cuando un RF se rechaza, el gestor lanza una excepción con el motivo y no cambia ningún dato: primero valida y después modifica. Los permisos de cada rol los define el menú de la consola según el tipo de Usuario autenticado, y los gestores reciben los tipos que cada operación necesita (reservar pide un Jugador, los turnos piden un AbstractEmpleado).

General

RF-GEN-01: GestorUsuarios.autenticar, que usa Usuario.autenticar y Usuario.estaActivo.

Administrador

RF-ADM-01 Consultar estados: ClubDeportivo.getCanchas (con Cancha.getOcupaciones) y ClubDeportivo.getArticulos (con Articulo.getCantidad por ubicación). Un inventario en cero se muestra vacío.

RF-ADM-02 Consultar historial: GestorReservas.getReservas, GestorReservas.getEntrenamientos y GestorDeportivo.getPartidos, los tres con desde y hasta. Rechazan un rango invertido.

RF-ADM-03 Registrar equipo: GestorDeportivo.registrarEquipo.

RF-ADM-04 Dar de baja equipo: GestorDeportivo.darDeBajaEquipo, que usa Equipo.darDeBaja y rechaza si el equipo está en una competencia en curso o tiene partidos por disputar (revisa getCompetencias y getPartidos del Club).

RF-ADM-05 Registrar jugador: GestorUsuarios.registrarJugador.

RF-ADM-06 Dar baja a un jugador: GestorUsuarios.darDeBajaJugador.

RF-ADM-07 Mover artículos: GestorVentas.moverArticulo, que usa Articulo.mover.

RF-ADM-08 Reabastecer tienda: GestorVentas.reabastecerTienda, que usa Articulo.reabastecer.

RF-ADM-09 Aprobar sugerencias: GestorVentas.resolverSugerencia. Si se aprueba, crea el Articulo o el Snack y lo agrega al Club; rechaza si ya está registrado (SP-03).

RF-ADM-10, RF-ADM-11 y RF-ADM-12 Turnos: GestorTurnos.asignarTurno, retirarTurno y resolverSolicitud.

RF-ADM-13 Reportes de ventas: GestorVentas.getVentas(rubro, desde, hasta). Los rubros son tienda, cafetería y mensualidades; el rubro lo da el tipo de la transacción. El período diario, semanal o mensual es el rango de fechas. Las mensualidades no tienen ningún RF que las cobre, así que ese rubro sale vacío (el RF acepta reportes vacíos).

RF-ADM-14 Reportes deportivos: asistencia con GestorReservas.getEntrenamientos, resultados con GestorDeportivo.getPartidos, y evolución de rankings con GestorDeportivo.getRanking e InscripcionDisciplina.getHistorialRanking.

RF-ADM-15 Crear competencia: GestorDeportivo.crearCompetencia.

RF-ADM-16 Inscribir equipo a competencia: GestorDeportivo.inscribirEquipoCompetencia, que usa AbstractCompetencia.inscribirEquipo.

Socio (Jugador)

RF-SOC-01 Inscribirse a un equipo: GestorDeportivo.inscribirEnEquipo, que usa Equipo.inscribirJugador.

RF-SOC-02 Inscribirse a una academia: GestorDeportivo.inscribirEnAcademia. Crea la InscripcionDisciplina en PRINCIPIANTE y se la agrega al jugador.

RF-SOC-03 Retirarse: GestorDeportivo.retirarseDeEquipo (Equipo.retirarJugador) y retirarseDeAcademia (Jugador.retirarInscripcion).

RF-SOC-04 Reservar una instalación: GestorReservas.reservar.

RF-SOC-05 Cancelar reserva: GestorReservas.cancelarReserva, que usa Reserva.cancelar y ClubDeportivo.liberarCancha.

RF-SOC-06 Programar un partido de práctica: GestorReservas.programarPractica. Después del partido, GestorReservas.registrarResultadoPractica llama a PartidoIndividual.actualizarRankings.

RF-SOC-07 Ranking por disciplina: GestorDeportivo.getRanking.

RF-SOC-08 Consultar inscripciones: GestorDeportivo.getEquiposDe y Jugador.getInscripciones.

RF-SOC-09 Estadísticas deportivas: GestorDeportivo.getEstadisticasDe.

RF-SOC-10 y RF-SOC-11 Convocatorias, asistencia y evaluaciones: GestorReservas.getEntrenamientosDe, y de cada Entrenamiento sus convocados (getConvocados), que traen el estado, la nota y el comentario.

RF-SOC-12 Comprar artículos: GestorVentas.comprarTienda.

RF-SOC-13 Comprar en cafetería: GestorVentas.comprarCafeteria.

RF-SOC-14 Consultar puntos: Usuario.getPuntosFidelidad.

RF-SOC-15 Redimir puntos: el parámetro puntos de comprarTienda y comprarCafeteria, que usa Usuario.redimirPuntos.

RF-SOC-16 Historial de compras: Usuario.getTransacciones.

Empleados (Entrenador y Fisioterapeuta)

Los RF-SOC-12 a RF-SOC-16 se resuelven igual para un empleado, con el 15% propio (AbstractEmpleado.getCodigoDescuento).

RF-EMP-01 Compartir código de descuento: GestorVentas.compartirCodigo.

RF-ENT-01 a RF-ENT-04 Entrenamientos: GestorReservas.crearEntrenamiento, cancelarEntrenamiento, registrarAsistencia y registrarEvaluacion.

RF-ENT-05 Registrar alineación: GestorDeportivo.registrarAlineacion.

RF-ENT-06 Registrar resultado: GestorDeportivo.registrarResultadoPartido. Antes, el entrenador registra las estadísticas de cada jugador con AbstractPartido.registrarEstadisticas. Los partidos se programan con GestorDeportivo.programarPartidoInterno y programarPartidoCompetencia.

RF-ENT-07 Consultar turnos: GestorTurnos.getTurnos. Los turnos son semanales (día de la semana), así que se consulta la semana completa y no hay un rango de fechas que pueda quedar invertido.

RF-ENT-08 Solicitar cambio de turno: GestorTurnos.solicitarCambioTurno.

RF-ENT-09 Sugerir nuevo producto: GestorVentas.sugerirArticulo.

RF-ENT-10 Consultar datos de jugadores: GestorReservas.getEntrenamientosDe y GestorDeportivo.getEstadisticasDe, rechazando al jugador que no le corresponde al entrenador.

RF-FIS-01, RF-FIS-02 y RF-FIS-03: los mismos de RF-ENT-07, RF-ENT-08 y RF-ENT-09 (SP-06).

Reglas del negocio: quién decide cada una

RN-01: GestorTurnos (cumpleCobertura).

RN-02: los tipos. Admin no es Jugador y los métodos de reservar, inscribir y practicar piden Jugador.

RN-03: GestorDeportivo.inscribirEnEquipo, que mira los equipos del jugador (getEquiposDe).

RN-04: Categoria.admiteEdad, que usa Equipo. RN-05: Equipo (camiseta única).

RN-06, RN-07 y RN-08: DepIndividual.admiteJugadores en Tenis, Padel y TenisMesa, y Cancha.tieneCapacidadPara.

RN-09: AbstractDeporte.maxSimultaneas junto con ClubDeportivo.contarOcupadas.

RN-10: Nivel.esAdyacente en GestorReservas.programarPractica, que no la aplica si alguno de los jugadores es también entrenador (Jugador.esTambienEntrenador).

RN-11: AbstractEmpleado.codigoDescuento y GestorVentas: 15% para el empleado, 8% para el socio con quien lo comparte, sin acumular.

RN-12: Cancha.permiteBebidaCaliente, usada en GestorReservas.reservar y en GestorVentas.comprarCafeteria.

RN-13: Snack.alergenos y SugerenciaArticulo.alergenos.

RN-14 y RN-15: TransCafeteria (IMPUESTO del 8% y propina). RN-16: la fecha de AbstractTransaccion y su tipo, que es el rubro. RN-17: Usuario.acumularPuntos. RN-18: TransTienda (IVA del 19%).

RN-19: DepConjunto.getMinJugadores en GestorDeportivo.registrarAlineacion.

RN-20: InfoJugador y JugadorNomina, aplicadas en GestorDeportivo.registrarResultadoPartido.

RN-21: EstadoAsistencia. RN-22: AsistenciaEntreno (nota de 1 a 5 con comentario opcional).

RN-23: la nómina de ParticipacionCompe en registrarAlineacion. RN-24: AbstractCompetencia.inscribirEquipo. RN-25: DepConjunto (puntos y permiteEmpate) junto con ParticipacionCompe.sumarPuntos.

Estado de la aplicación

El estado de la aplicación es el ClubDeportivo y todo lo que cuelga de él. No hay otro lugar donde se guarde nada, y eso hace que el estado completo se pueda guardar, cargar y revisar desde un solo objeto. Qué estado se representa y dónde vive:

Usuarios (socios, empleados y administrador): en Club.usuarios, con login, clave, estado activo y puntos de fidelidad. Dar de baja es poner activo en falso; no se borra nada (SP-01).

Equipos e inscripciones: Club.equipos con el mapa de jugadores y su InfoJugador (posición, camiseta y suspensión), y Jugador.inscripciones con el nivel y el ranking de cada academia.

Alquiler de instalaciones: Club.canchas, cada una con sus Ocupaciones, y Club.reservas y Club.entrenamientos, cada uno con su cancha y si sigue activo. Una cancha ocupada es una Ocupacion que se agrega y se quita al cancelar.

Competencias y partidos: Club.competencias con sus participaciones, nóminas y fases, y Club.partidos con el marcador, la alineación y las estadísticas por jugador. Las suspensiones viven en InfoJugador y JugadorNomina.

Entrenamientos: cada uno con sus convocados, y por convocado el estado de asistencia, la nota y el comentario.

Ventas e inventario: Club.articulos con la cantidad en cada ubicación (bodega, tienda y cafetería), y en cada Usuario sus transacciones con sus compras y sus puntos de fidelidad. Las sugerencias están en Club.sugerencias con su estado.

Estados financieros: no hay una clase de contabilidad aparte. Salen de las transacciones: cada una guarda su fecha, su rubro (por el tipo), el descuento, el impuesto o IVA, la propina y el total, y GestorVentas.getVentas las reune por rubro y período.

Turnos: Club.turnos y Club.solicitudes, estas con su estado.

El estado solo cambia a través de los gestores y de los objetos que ellos manipulan; las consolas no tocan las listas del Club. Las consolas de prueba (P1 a P9) imprimen antes y después de cada operación la parte del estado que toca, por ejemplo, las ocupaciones de una cancha o la cantidad de un artículo en cada ubicación. En un rechazo imprimen el motivo y muestran que el estado no cambió.

Persistencia

Al final no pudimos implementar la persistencia, pero igual tuvimos una idea, no fue por culpa de la dificultad sino por irresponsabilidad y no alcanzar en tema de tiempo. Según nuestra idea de persistencia toda la información es persistente. Pensamos en guardarla en archivos JSON (texto plano), dentro de una carpeta datos que está fuera de la carpeta del código fuente (al lado de src, no dentro). Para organizar el código seguimos la estructura que usamos en el taller de la aerolínea: una interfaz por cada tipo de información, una implementación en JSON de cada una y una clase central que entrega la implementación según el formato.

Cómo iba a funcionar

Todo iba en el paquete persistencia, con estas clases:

- **IPersistenciaClub: define cargarClub(archivo, club) y salvarClub(archivo, club). Se encarga de la estructura del club: el horario de atención, los deportes con sus reglas, las canchas, los artículos con su cantidad en cada ubicación, los usuarios y los equipos. Su archivo es club.json.**
- **IPersistenciaOperacion: define cargarOperacion(archivo, club) y salvarOperacion(archivo, club). Se encarga de lo que pasa día a día: reservas, entrenamientos con su asistencia, competencias con sus partidos y estadísticas, transacciones con sus compras, turnos y solicitudes. Su archivo es operacion.json.**

- **PersistenciaClubJson y PersistenciaOperacionJson: implementan las interfaces con la librería org.json. Al salvar, recorren el club, arman un JSONObject con JSONArray para cada lista y lo escriben en el archivo. Al cargar, leen el archivo completo, crean los objetos y los agregan al club con los mismos métodos que usan los gestores.**
- **CentralPersistencia: es la fábrica. Tiene la constante JSON y los métodos getPersistenciaClub(tipo) y getPersistenciaOperacion(tipo). Si el tipo no existe lanza TipoInvalidoException. Así, si algún día se quiere otro formato, se agrega otra implementación y no se toca ni la consola ni los gestores.**
- **TipoInvalidoException e InformacionInconsistenteException: la primera se lanza con un formato desconocido y la segunda cuando el archivo apunta a algo que no existe.**

Se carga primero club.json y después operacion.json, porque el segundo apunta a objetos del primero.

En JSON no existen las referencias entre objetos, así que cada objeto al que otro apunta se guarda con su identificador: el id de la cancha, el nombre del equipo, del artículo y de la competencia, y el login de los usuarios. Al cargar, se busca ese identificador en el club y se enlaza el objeto. Si no aparece, se lanza InformacionInconsistenteException.

Cuándo: la consola carga al arrancar (primero el club y luego la operación) y guarda después de cada operación que cambió datos, para que no se pierda lo hecho si se cierra de golpe. Los gestores se crean de nuevo con el club cargado. Si todavía no hay archivos, se crea el club inicial con los objetos que ningún RF crea: el horario de atención, los seis deportes con sus reglas, las canchas del club, el administrador y los artículos base.

Qué se guarda y qué no

- Se guarda todo el estado de la sección 6, repartido entre los dos archivos.
- No se guardan los gestores: no tienen datos, solo el club que reciben en el constructor.
- Las constantes IVA e IMPUESTO son static final, viven en el código y no se guardan.
- Las fechas son String y las horas son int, así que en el JSON quedan como texto y número. Las enumeraciones (Categoria, Nivel, EstadoAsistencia, Ubicacion, EstadoSolicitud y DayOfWeek) se guardan como texto con su nombre y se recuperan con valueOf.
- Las clases abstractas (deportes, estadísticas, transacciones, competencias y partidos) se guardan con un campo tipo, para saber qué subclase crear al cargar. Es la misma idea de los clientes naturales y corporativos del taller.
- El login y la clave se guardan tal cual. El enunciado supone que solo la aplicación lee y escribe esa carpeta.

Por qué JSON y por qué dos archivos

El enunciado permite archivos planos o binarios, en una carpeta que no sea la del código, y pide pensar cuántos archivos hacer. Escogimos JSON porque es texto plano que se puede abrir y revisar, y porque es la forma de persistencia que ya trabajamos en el curso. Escogimos dos archivos porque son dos tipos de

información distintos: la estructura del club casi no cambia, y la operación cambia todo el tiempo. Como la operación depende de la estructura, se cargan en ese orden. Un solo archivo habría mezclado ambas cosas, y uno por clase habría obligado a enlazar demasiadas referencias al cargar.

Lo que implica:

- Hay más código que con la serialización de Java, porque cada clase tiene que armarse y reconstruirse a mano, y cada objeto referenciado necesita un identificador.
- Cada guardado reescribe los archivos completos. Con el tamaño de un club universitario no es un problema.
- Si se agrega o cambia un atributo de una clase, hay que actualizar también su parte de salvar y de cargar.
- La carpeta datos queda fuera de src para que el código y los datos no se mezclen y limpiar el proyecto no borre los datos. Los archivos de prueba que se entregan son el club.json y el operacion.json que genera la propia aplicación con datos de ejemplo.

serialización esas referencias se conservan solas y, al cargar, el mismo Jugador sigue siendo un solo objeto. Con varios archivos de texto habría que inventar identificadores y volver a enlazar todo al cargar, con más código y más posibilidades de dejar datos inconsistentes.

Lo que implica:

- El archivo no se puede leer a mano. Para revisar los datos se usan las consolas.
- Cada guardado reescribe todo el club. Con el tamaño de un club universitario no es un problema.
- Si se cambia una clase serializable, hay que volver a generar el archivo de prueba.
- La carpeta datos queda fuera de src para que el código y los datos no se mezclen y limpiar el proyecto no borre los datos. El archivo de prueba que se entrega es el club.dat que genera la propia aplicación con datos de ejemplo.

Diagramas de clases, normal y de alto nivel

Los Diagramas de clase y normal los realizamos con a la información recopilada en este documento, al final los 2 diagramas están relacionados, ya que el de clases es el normal, el definitivo, la guía para poder crear e implementar todas las funciones en java, mientras el de alto nivel es uno que este hecho para facilitar el entendimiento de todo el proyecto. Los 2 diagramas están en el archivo del repositorio, y al ser tan grandes, es casi inútil agregarlos acá.

Diagramas de secuencia

Para los diagramas de secuencia Utilizamos un pequeño Prompt con Claude sonnet 5.5, porque la verdad estábamos algo confundidos en cómo hacer los diagramas, por lo que le pedimos que nos ayudara a entender y nos diera un paso a paso para hacerlos:
Prompt: “Podrías darnos una pequeña guía o paso a paso para poder realizar los diagramas de secuencia del siguiente proyecto en UMLet, porfa danos una guía, no necesitamos que los hagas, sino que nos

digas más o menos como hacerlos con la siguiente información: (info de las clases y el diagrama original)". Respuesta: **Paso a paso (como diseñador en UMLet)**

1. Escribe el guion antes de dibujar. Para cada RF, en texto plano, pon quién empieza y qué llama a qué, en orden. Ejemplo de turnos:

- El administrador llama a resolverSolicitud del GestorTurnos.
- El gestor pide las solicitudes al Club y mira el estado.
- Si no está PENDIENTE, lanza excepción.
- Si se aprueba, aplica el cambio, revisa la cobertura y, si no cumple, lo deshace.

Cada línea del guion se convierte en una flecha.

2. Elige los participantes. Solo los objetos que intervienen, de izquierda a derecha. Primero el actor que inicia, después el gestor (el controlador), después el resto en el orden en que aparecen. Cada uno se dibuja como una caja con nombre:Clase, por ejemplo gt:GestorTurnos.

3. Arma el marco. En la paleta "UML Sequence" de UMLet, arrastra un interaction frame y ponle el título (RF-ADM-12 Aprobar o rechazar un cambio de turno). Pon las cajas de los participantes arriba, alineadas.

4. Baja las líneas de vida. Cada caja lleva una línea punteada vertical (una relación con lt=.) que baja hasta el final del marco.

5. Dibuja las flechas de arriba hacia abajo. Cada llamada es una flecha de punta rellena (lt=->>>), con el nombre del método encima. Cada respuesta es una flecha punteada de punta abierta (lt=.>) que vuelve. Una creación es una flecha con "new" hacia una caja que aparece más abajo. Mantén un espacio parejo entre flechas.

6. Pon las barras de activación. Es un rectángulo blanco angosto sobre la línea de vida, desde que recibe la llamada hasta que responde.

7. Agrega los marcos de decisión. Aquí es donde los pasos 1 y 2 se vuelven lógica:

- Una validación que rechaza se dibuja con un marco opt, con la condición entre corchetes y la flecha "lanza excepción" adentro.
- Un si/si no usa un marco alt con una línea punteada en medio.
- Algo repetido usa un marco loop.

Se dibuja el marco encima de las flechas que cubre y se escribe la condición como texto sobre una línea de vida.

8. Revisa contra las clases. Cada nombre de método en una flecha debe existir en el diagrama de clases, en la clase de la caja que lo recibe. Si no existe, o falta el método, o la flecha está mal.

9. Revisa el orden. Primero validar y después cambiar cosas, porque así quedó el diseño. La excepción es el cambio de turno, que aplica, revisa y deshace.

Escogimos 4 RF críticos porque son los que cruzan más objetos y reúnen más reglas: reservar una instalación (RF-SOC-04), comprar en la cafetería (RF-SOC-13), registrar el resultado de un partido (RF-ENT-06) y aprobar o rechazar un cambio de turno (RF-AMD-12). Solo aparecen los llamados que importan, los getters simples no. Todos los métodos que se ven están en el diagrama detallado.

Uso de IA:

Se utilizó la inteligencia artificial únicamente para tener claro el esqueleto del proyecto en código, de acuerdo con lo definido por las decisiones del modelo y el diagrama creado. Esto incluye solo partes del código que son sencillamente mecánicas de escribir (atributos que ya se tenían claros o getters de una sola línea